[root@llm:~]#

PROMPT INJECTION.

// what it is · why · papers · standards · tools · defenses · the future

```
> cat ./threat-model.txt
OWASP LLM01:2025  →  #1 LLM risk
MITRE ATLAS       →  AML.T0051
Status            →  unsolved at the model layer (probably forever)
```

$ what is prompt injection?

An attack where **untrusted input** is interpreted by an LLM as **instructions** rather than data.

PROMPT INJECTION

Subverts the **application** built on top of an LLM.

Goal: ignore system prompt, leak data, misuse tools.

Coined by Simon Willison, Sept 2022.
Named after SQL injection.

JAILBREAKING

Bypasses the **model's safety alignment.**

Goal: get refused content ("how do I make X").

Different threat model.
Different defenses.
Often confused with injection.

// flavors: ***direct*** *(user types it)* · ***indirect*** *(hidden in retrieved content — emails, web pages, RAG docs, tool output)*

$ why it happens

// transformers process one undifferentiated token stream.

```
trusted_prompt  +  untrusted_input  ──▶  model.attention()  ──▶  ??

// no privilege bit on tokens.
// no syntactic boundary the model is forced to respect.
// the most recent / most persuasive instruction tends to win.
```

> ***"Security requires boundaries, but LLMs dissolve boundaries."***
> — Simon Willison

$ the lethal trifecta

// an agent is exploitable when it has all three of:

PRIVATE DATA

emails
documents
databases
secrets

UNTRUSTED CONTENT

web pages
emails
tool output
RAG documents

EXFIL CHANNEL

outbound URLs
image renders
API calls
tool actions

all three present ——▶ **zero-click exfil possible.** remove one leg or accept the risk.

$ incidents in the wild

2022	remoteli.io Twitter bot	GPT-3 customer-facing bot hijacked by tweet replies.
2023	Bing Chat / Auto-GPT	Indirect injection, autonomous agent RCE.
2025	GitHub Copilot CVE-2025-53773	RCE via prompt injection. CVSS 9.6.
2025	CamoLeak ("zero-click")	Crafted email → enterprise agent → data exfil.
2025	Cline npm cache poisoning	Issue-title injection → typosquat install → ~4k devs backdoored.
2025	PoisonedRAG	5 docs in millions → 90% manipulation rate.

$ papers / required reading

ATTACKS

- » Perez & Ribeiro 2022 — PromptInject
- » Greshake et al. 2023 — Indirect injection
- » Liu et al. 2023 — HouYi (arXiv:2306.05499)
- » Zou et al. 2023 — GCG suffixes
- » Abdelnabi et al. 2025 — LLMail-Inject

DEFENSES

- » Wallace et al. 2024 — Instruction Hierarchy
- » Hines et al. 2024 — Spotlighting (2403.14720)
- » Debenedetti et al. 2025 — CaMeL (2503.18813)
- » Beurer-Kellner et al. 2025 — Design Patterns
- » Costa et al. 2025 — IFC for AI agents

$ standards (the de-facto RFCs)

// no IETF RFC for prompt injection. these are what the industry uses:

OWASP LLM Top 10 (2025)	LLM01 prompt injection · LLM07 system prompt leakage · LLM08 vector/embedding weaknesses
MITRE ATLAS	16 tactics · 84 techniques · headline = AML.T0051 (next slide)
NIST AI Risk Management Framework	general AI risk; scanners map findings here
EU AI Act	compliance framework; promptfoo/DeepTeam map findings
OWASP GenAI Security Project	testing methodology + red-teaming guide

$ MITRE ATLAS · the kill chain

// a single injection incident is never just T0051. test the chain.

AML.T0043
Craft Adversarial Data

→

AML.T0051
LLM Prompt Injection

→

AML.T0086
Exfil via Agent Tool

→

AML.T0024
Exfil via Inference API

// Oct 2025 + Zenity Labs collab — agent-specific additions:

Context Poisoning · Memory Manipulation · Thread Injection · Modify Agent Config · RAG Credential Harvesting · Publish Poisoned AI Agent Tool

$ tools / red team kit

$ garak

NVIDIA AI Red Team

120+ probes. nmap-for-LLMs. AVID integration.

$ promptfoo

OpenAI (formerly indep.)

context-aware fuzzing. CI-friendly. ATLAS-mapped.

$ PyRIT

Microsoft AI Red Team

multi-turn (Crescendo, TAP). policy-driven.

$ FuzzyAI

open source

genetic-algo mutation. ArtPrompt, Unicode smuggling.

$ Augustus

Praetorian (2026)

Go binary. 210+ probes. no Python deps.

$ Lakera Guard

commercial

real-time API. + Gandalf CTF for learning.

$ defense in depth (5 layers)

L1	Hardened prompts + spotlighting	marker-wrapped untrusted input. probabilistic.
L2	Detection classifiers	Lakera Guard / Prompt Shields / activation-based.
L3	Dual-LLM pattern (Willison)	privileged LLM never sees untrusted content.
L4	CaMeL / IFC (the architectural fix)	capabilities + custom interpreter. provable.
L5	Break the lethal trifecta	remove one leg. only defense Willison trusts.

// strength ▲ as you go down

$ CaMeL · defeating injection by design

// Google DeepMind + ETH Zürich · arXiv:2503.18813 · March 2025

```
user query ──▶ P-LLM ──▶ DSL program ──▶ interpreter ──▶ tools
                                  │
                                  ▼
                              Q-LLM  (sees untrusted text, no tools)
```

every value carries a **capability** · interpreter enforces policy at every tool call

AgentDojo benchmark: **77%** of tasks solved with **provable security** · near-100% block rate vs. attacks · cost: -7pp utility vs. undefended

$ pre-prod checklist

[01] Map data flows. If your agent has all 3 trifecta legs → redesign.

[02] Run garak / promptfoo in CI. Treat regressions as blockers.

[03] Add an input/output classifier (Prompt Shields / Lakera / LLM Guard).

[04] Apply spotlighting + structured prompts. Cheap, slightly helpful.

[05] High-stakes? Use dual-LLM or CaMeL-style architecture.

[06] Require human confirmation for irreversible actions.

[07] Plant canary tokens in private data. Alert on outbound matches.

$ will it ever be fixed?

not at the model layer.

contained at the system layer.

ARCHITECTURAL

no privilege bit on tokens. transformers can't tell instructions from data.

PROBABILISTIC

99% blocking is a failing grade in security. attackers iterate.

STATEFUL

once poisoned, context propagates. memory becomes a liability.

"Once an LLM agent has ingested untrusted input, it must be constrained so that input cannot trigger consequential actions." — Willison

$ links / further reading

→ OWASP LLM Top 10 (2025)	genai.owasp.org/llmrisk/llm01-prompt-injection/
→ Simon Willison · prompt-injection tag	simonwillison.net/tags/prompt-injection/
→ The lethal trifecta	simonwillison.net/2025/Jun/16/the-lethal-trifecta/
→ CaMeL paper (DeepMind)	arxiv.org/abs/2503.18813
→ MITRE ATLAS	atlas.mitre.org
→ garak (NVIDIA)	github.com/NVIDIA/garak
→ promptfoo	github.com/promptfoo/promptfoo
→ Gandalf (CTF — best hands-on intro)	gandalf.lakera.ai

$ exit_